

Load Studio

PWA Cinema Experience

Part 2 and Complete Start-to-Finish Build Manual

For a YouTube-like and Tubi-like platform built around custom PWA, HTML, and video content.

Load Studio: PWA Cinema Experience

Part 2 and Complete Start-to-Finish Build Manual

Prepared as a developer-ready manual for turning Load into a PWA-native cinema creation and distribution system.

This manual assumes Load already has working editor features inspired by VN app workflows, sound effects, character voices, import and export options, recording, voice changer features, PWA, HTML, and digital book creation tools and export, a video player, and a working AI image generator that is still being expanded toward Glam AI chat features.

The goal is not to rebuild Load. The goal is to add the Load Studio layer on top of what already works.

Table of Contents

1. Product Thesis
2. What Already Exists in Load
3. What Load Studio Adds
4. The New Format: Load Studio Package
5. Platform Strategy: YouTube-like plus Tubi-like
6. Start-to-Finish Build Roadmap
7. Package Structure
8. Scene JSON
9. Character Bible JSON
10. Rights Metadata

11. Player Engine
12. Editor Integration
13. AI Image and Glam AI Chat Integration
14. Export and Reopen System
15. Platform Upload and Safety
16. Streaming Experience
17. Optional Video Export
18. Local Engine Future
19. Starter Code
20. Developer Prompt
21. Final Build Checklist

1. Product Thesis

Load Studio becomes a PWA-native cinema studio. It lets creators turn books, scripts, digital books, HTML projects, images, narration, music, and sound into playable PWA cinema packages.

The viewer experiences the result like video. The creator experiences it like an editable project.

This is the strong category:

```
PWA-native cinema creation and distribution.
```

The strategic sentence:

```
Load Studio lets creators turn books, scripts, and visual stories into playable PWA cinema packages that stream like video, remain editable like projects, and can later export to video or publish directly to a dedicated PWA-native streaming platform.
```

2. What Already Exists in Load

Do not rebuild these systems from scratch. Reuse and extend them.

- VN-style editor features
- Sound effects
- Character voices
- Import and export options
- Recording

- Voice changer features
- PWA creation tools
- HTML creation tools
- Digital book creation tools and export
- Video player
- AI image generator
- Developing Glam AI chat features

3. What Load Studio Adds

Load Studio adds a new project type:

```
Load Studio Cinema Project
```

This project type should support:

- Scene cards
- Timeline playback
- Imported or generated images
- Narration
- Character voices
- Subtitles
- Music
- Sound effects
- Title cards
- Credits
- Offline PWA playback
- Export as a Load Studio package
- Reopen and edit inside Load
- Later export as MP4 or upload to the platform

4. The New Format: Load Studio Package

A Load Studio package is a structured zip file. It may later use a custom extension, but the MVP can use:

```
.loadstudio.zip  
.cinepwa.zip
```

It is not just a video. It is a coded film package containing player files, scene data, character data, rights metadata, and assets.

5. Platform Strategy: YouTube-like plus Tubi-like

The platform should have two sides.

Creator side, YouTube-like

- Creator upload
- Creator channels
- Drafts and published projects
- PWA and HTML package support
- MP4 support
- Project analytics later
- Creator monetization later

Viewer side, Tubi-like

- Featured rows
- Newly Added
- Originals
- Documentaries
- Reality
- Biographies
- Animations
- Kids
- News
- Book Cinema
- PWA Originals
- Public domain classics

The viewer should not need to know whether the content is MP4, HTML, or PWA. The viewer presses play.

6. Start-to-Finish Build Roadmap

Phase 1: Define the format

Create the Load Studio Package Spec v0.1.

Phase 2: Build the player

Make one package open and play like a small film.

Phase 3: Connect the editor

Use the existing editor to create and revise scene cards.

Phase 4: Add export and reopen

Export a package. Reopen the same package inside Load.

Phase 5: Connect AI

Use existing AI image generation and future Glam AI chat features to create scene images, prompts, summaries, captions, and consistency checks.

Phase 6: Add platform validation

Validate uploaded packages and block unsafe content.

Phase 7: Build the streaming shell

Add browsing rows, creator pages, search, categories, and continue watching.

Phase 8: Add video export later

Add MP4 export through MediaRecorder, FFmpeg.wasm, backend FFmpeg, or Load Local Engine.

7. Package Structure

```
My_Project.loadstudio.zip
  index.html
  manifest.json
  service-worker.js
  project.json
  scenes.json
  characters.json
  rights.json
  credits.json
  styles.css
  player.js
  editor.js
  assets/images/
  assets/audio/
  assets/music/
  assets/sfx/
  assets/subtitles/
  assets/icons/
```

8. Scene JSON

Each scene should be stored as structured data.

```
{
  "sceneId": "scene_001",
  "chapter": "Chapter 1",
  "title": "The Story Begins",
  "summary": "Opening scene summary.",
  "characters": ["Narrator"],
  "location": "Opening title card",
  "mood": "quiet anticipation",
  "duration": 12,
  "narrationScript": "This is where the visual story begins.",
  "dialogueNotes": "",
  "imagePrompt": "cinematic story opening, dramatic light, premium title card",
  "negativePrompt": "distorted text, blurry faces, modern objects unless requested",
  "subtitleText": "This is where the visual story begins.",
  "image": "assets/images/scene_001.png",
  "audio": "assets/audio/scene_001_voice.mp3",
  "music": "assets/music/scene_001_music.mp3",
  "sfx": [],
  "camera": {
    "type": "ken_burns",
    "startScale": 1.05,
    "endScale": 1.18,
    "panX": -2,
    "panY": -1
  },
  "transitionIn": "fade",
  "transitionOut": "crossfade",
  "status": "draft"
}
```

9. Character Bible JSON

```
{
  "characterId": "character_001",
  "name": "Character Name",
  "ageRange": "adult",
  "appearance": "locked description",
  "hair": "locked hair description",
  "clothing": "locked clothing description",
  "voiceStyle": "warm narrator, grave elder, child voice, etc.",
  "referenceImages": ["assets/images/character_reference.png"],
  "consistencyRules": "Preserve face, clothing, era, and visual style.",
  "negativeTraitsToAvoid": "do not alter identity, do not modernize wardrobe"
}
```

10. Rights Metadata

Rights metadata must be part of the package from the beginning.

```
{
  "owner": "Creator name or company",
  "license": "user-owned, public-domain, licensed, or platform-original",
  "sourceMaterial": "Book, script, public domain source, or original manuscript",
  "assetDeclarations": [
    { "asset": "assets/images/scene_001.png", "status": "user-generated" },
    { "asset": "assets/audio/scene_001_voice.mp3", "status": "user-recorded" }
  ],
  "notes": "Creator confirms rights to upload and distribute."
}
```

11. Player Engine

The player should make a scene package play like a film.

Features:

- Scene image background
- Pan and zoom motion
- Fade and crossfade transitions
- Narration audio
- Subtitles
- Music and sound effects
- Play, pause, restart
- Next and previous scene
- Progress bar
- Offline service worker support

Core playback logic:

```
const state = {
  project,
  sceneIndex: 0,
  playing: false,
  timer: null,
  audio: null
};

function currentScene() {
  return state.project.scenes[state.sceneIndex];
}

function renderScene() {
  const scene = currentScene();
  sceneTitle.textContent = scene.title;
  subtitleText.textContent = scene.subtitleText || '';
  sceneImage.style.backgroundImage = `url('${scene.image}')`;
  sceneImage.style.setProperty('--duration', `${scene.duration || 10}s`);
}

function playScene() {
  const scene = currentScene();
  state.playing = true;
  renderScene();
  if (scene.audio) {
```

```
    state.audio = new Audio(scene.audio);
    state.audio.play().catch(() => {});
  }
  state.timer = setTimeout(nextScene, (scene.duration || 10) * 1000);
}

function nextScene() {
  clearTimeout(state.timer);
  if (state.audio) state.audio.pause();
  state.sceneIndex = Math.min(state.project.scenes.length - 1, state.sceneIndex + 1);
  playScene();
}
```

12. Editor Integration

Because Load already has VN-style editor features, Load Studio should be added as a new mode inside the existing editor.

Required integration points:

- Scene cards connect to timeline items
- Existing image generator attaches output to scene.image
- Existing sound effects attach to scene.sfx
- Existing character voices attach to scene.audio
- Existing recorder creates narration clips
- Existing voice changer can process narration or character speech
- Existing import/export creates and opens package zip files
- Existing video player gains a scene playback mode

13. AI Image and Glam AI Chat Integration

AI should assist, not block the workflow.

AI tasks:

- Scene summary
- Scene split
- Character extraction
- Visual prompt
- Negative prompt
- Shot list
- Subtitle draft
- Narration draft
- Image generation
- Image consistency check

- Glam AI chat style creative guidance

Provider rule:

Never claim image generation worked unless a real image file, blob, or URL is returned.

Provider routing example:

```
export async function runSceneImageRequest({ scene, providers }) {
  for (const provider of providers) {
    if (!provider.capabilities?.imageGeneration) continue;
    try {
      const result = await provider.generateImage({
        prompt: scene.imagePrompt,
        negativePrompt: scene.negativePrompt
      });
      if (result?.imageUrl || result?.blob || result?.file) {
        return { ok: true, provider: provider.name, result };
      }
    } catch (error) {
      console.warn(provider.name, error);
    }
  }
  return { ok: false, fallbackPrompt: scene.imagePrompt };
}
```

14. Export and Reopen System

Use JSZip for browser-side package creation.

```
import JSZip from 'jszip';

export async function buildLoadStudioZip(project, assets = []) {
  const zip = new JSZip();
  zip.file('project.json', JSON.stringify(project, null, 2));
  zip.file('scenes.json', JSON.stringify(project.scenes || [], null, 2));
  zip.file('characters.json', JSON.stringify(project.characters || [], null, 2));
  zip.file('rights.json', JSON.stringify(project.rights || {}, null, 2));

  for (const asset of assets) {
    zip.file(asset.path, asset.blob || asset.text || '');
  }
  return await zip.generateAsync({ type: 'blob' });
}
```

15. Platform Upload and Safety

If creators upload HTML or PWA packages, the platform must validate and sandbox them.

Do not allow random HTML to run freely.

Safety requirements:

- Validate project.json
- Validate scenes.json
- Require rights.json
- Block unsafe external scripts
- Block hidden redirects
- Block credential capture forms
- Serve creator packages in sandboxed player
- Add Content Security Policy
- Limit external network calls
- Scan file names and asset paths

Validator example:

```
export function validateCinePwaPackage(project) {
  const errors = [];
  if (!project || typeof project !== 'object') errors.push('Project must be an object.');
```

```
  if (project.projectType !== 'loadstudio.cinepwa') errors.push('Invalid projectType.');
```

```
  if (!Array.isArray(project.scenes)) errors.push('Project must include scenes array.');
```

```
  if (!project.rights) errors.push('Project must include rights metadata.');
```

```
  return { valid: errors.length === 0, errors };
}
```

Recommended CSP model:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' blob: data;;
  media-src 'self' blob;;
  connect-src 'none';
  frame-ancestors 'self';
  base-uri 'none';
  form-action 'none';
```

16. Streaming Experience

The streaming platform should support:

- MP4 video
- WebM video
- Load Studio packages
- HTML cinema packages
- Audio story packages
- Public domain content
- Original branded content
- Creator-uploaded content after validation

Viewer rows:

- Featured
 - Newly Added
 - Originals
 - Book Cinema
 - PWA Originals
 - Documentaries
 - Reality
 - Biographies
 - Animations
 - Kids
 - News
-

17. Optional Video Export

MP4 export should not be the first dependency. The native format is the editable PWA cinema package.

Future export routes:

- Canvas captureStream plus MediaRecorder
- FFmpeg.wasm for short lightweight exports
- Backend FFmpeg for reliable export
- Load Local Engine for creator-owned desktop rendering

Browser recording example:

```
async function recordCanvasToWebM(canvas, durationMs = 10000) {
  const stream = canvas.captureStream(30);
  const recorder = new MediaRecorder(stream, { mimeType: 'video/webm' });
  const chunks = [];
  recorder.ondataavailable = event => chunks.push(event.data);
  recorder.start();
  await new Promise(resolve => setTimeout(resolve, durationMs));
  recorder.stop();
  await new Promise(resolve => recorder.onstop = resolve);
  return new Blob(chunks, { type: 'video/webm' });
}
```

18. Local Engine Future

The Local Engine allows heavy rendering without forcing paid cloud GPU use.

Possible tools:

- ComfyUI
- FFmpeg
- Piper or Kokoro TTS

- IOPaint or LaMa Cleaner
- SAM 2
- Real-ESRGAN
- ControlNet
- IP-Adapter

Local render endpoint example:

```
POST http://localhost:8787/render
{
  "projectType": "loadstudio.cinepwa",
  "project": { "title": "My Film", "scenes": [] },
  "output": "mp4",
  "resolution": "1280x720"
}
```

19. Starter Code

A starter code scaffold is included in the zip package. It includes:

- index.html
- styles.css
- player.js
- editor.js
- project.example.js
- manifest.json
- service-worker.js
- cinepwa.schema.json
- upload-validator.js
- security.csp.example.txt
- package.json

20. Developer Prompt

BUILD LOAD STUDIO: PWA CINEMA EXPERIENCE AND CREATOR PLATFORM FOUNDATION

Load already has working editor features inspired by VN app workflows, sound effects, character voices, import/export options, recording, voice changer features, PWA/HTML/digital book creation tools and export, a video player, and a working AI image generator that is still being expanded with Glam AI chat features.

Do not rebuild these systems from scratch. Extend them.

Goal:

Create Load Studio, a PWA-native cinema creation system where books, scripts, HTML projects, digital books, and creator assets can become playable standalone PWA/HTML cinema packages. These packages should open like video experiences, remain editable as structured projects, and later export to video files or upload to our YouTube-like and Tubi-like platform for HTML/PWA creators.

Core deliverable:

Build a working MVP that can:

1. Create a Load Studio project.
2. Build scene cards.
3. Attach images, narration, subtitles, music, and sound effects to scenes.
4. Play the scenes like a small film using the existing Load player foundation.
5. Save and export as a Load Studio package.
6. Reopen the package inside Load for editing.
7. Prepare the package for future upload to the platform.

Required package structure:

- index.html
- manifest.json
- service-worker.js
- project.json
- scenes.json
- characters.json
- rights.json
- credits.json
- styles.css
- player.js
- editor.js
- assets/images/
- assets/audio/
- assets/music/
- assets/sfx/
- assets/subtitles/
- assets/icons/

Build rules:

- Preserve existing Load features.
- Use existing editor, audio, voice, import/export, digital book, video player, and AI image generator where possible.
- Do not make MP4 export the MVP blocker.
- Make the PWA cinema package the native format.
- Add MP4 export later through MediaRecorder, FFmpeg.wasm, backend FFmpeg, or Load Local Engine.
- For platform upload, validate and sandbox packages.
- Do not allow arbitrary unsafe HTML uploads to run freely.
- Require rights metadata before publishing.
- Keep creator packages editable.
- Keep viewer playback simple: press play and watch.

Final report must include:

- files changed
- new modules added
- how to create a package
- how to reopen a package
- how playback works
- what is MVP complete
- what is future work
- any security risks remaining

21. Final Build Checklist

- Add Load Studio project type
- Add package schema and validator
- Add scene cards linked to the current editor timeline
- Attach existing audio, voice, SFX, recording, and voice changer tools to scenes
- Attach current AI image generator to scene prompts

- Add CinePWA player mode to current video player
- Add package export and import using JSZip
- Add standalone PWA export with manifest and service worker
- Add rights and credits metadata
- Add platform upload validator
- Add sandboxed viewer for creator packages
- Add MP4 export as a future path, not the MVP blocker

Closing Direction

Build the format first. Build the player second. Build the platform third.

That order protects the invention.